Data Structures & Algorithms

Student Hub | Original study summary | For revision and self-study

Course

CS301 | Student Hub Original Revision Guide

How to use this guide

Use each structure to explain the operation, its cost, and when it is a good choice. Verify complexity assumptions against the exact implementation.

1. Measuring algorithms

Big-O describes how an upper bound grows with input size. Common orders are O(1), O(log n), O(n), O(n log n), O(n^2), and O(2^n). State whether you mean time or auxiliary space.

2. Arrays and linked lists

Arrays support indexed access in O(1); inserting in the middle usually shifts elements and costs O(n). Linked lists insert at a known node in O(1), but finding a position takes O(n).

3. Stacks, queues, and hashing

A stack is last-in, first-out; a queue is first-in, first-out. A hash table has expected O(1) lookup with a good hash and controlled load factor, but collisions and resizing affect its worst case.

4. Trees and graphs

A balanced search tree supports search, insert, and delete in O(log n); an unbalanced tree can degrade to O(n). Breadth-first search uses a queue; depth-first search uses a stack or recursion. Both visit vertices and edges in O(V + E) with adjacency lists.

5. Sorting and selection

Merge sort runs in O(n log n) time and needs extra array space. Quicksort is commonly O(n log n) but has O(n^2) worst-case time. Choose stable sorting when equal-key order must be preserved.

Study checklist

For each problem: define input and output, choose a structure, write the invariant, estimate time and space, then test empty, one-item, duplicate, and large inputs.

Data Structures & Algorithms

Student Hub | Original study summary | For revision and self-study

Practice prompts

1) When is a linked list preferable to an array? 2) Why can a hash table have a worst case above O(1)? 3) Which traversal finds shortest paths in an unweighted graph? 4) What property makes a sorting algorithm stable?

Suggested answers

1) When frequent insertions/removals occur at known nodes and random indexing is not needed. 2) Collisions can collect many keys in one bucket. 3) Breadth-first search. 4) Equal-key items keep their original relative order.

Before an exam

Trace one stack, queue, tree rotation, graph traversal, and sort by hand. For each trace, label the data structure state after every operation.

Further reading

MIT OpenCourseWare 6.006 and the OpenDSA project are linked from the Student Hub Course Materials page.